

ATELIER SCIPY

Traitement d'Images avec scipy.ndimage

Digital School of Paris

Mastère Data Engineering — Promotion 2024-2026

Auteur : Franck Ulrich BIPANDA

Février 2026

Objectif de l'atelier

Manipuler la bibliothèque SciPy pour appliquer des techniques de traitement d'images à l'aide du module : `from scipy import ndimage`

Durée : 40 min

Travail individuel

Livrables : script + compte-rendu



Image utilisée pour l'atelier : deux Cane Corsos (612 × 408 px, RGB)

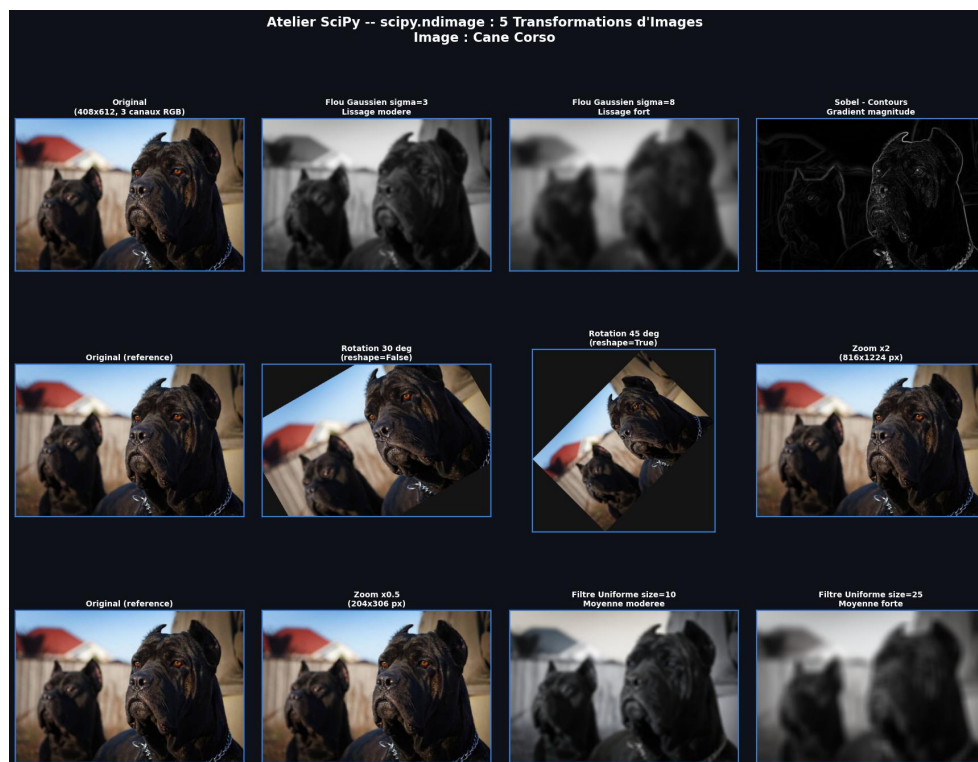


Figure 0 — Vue d'ensemble des 5 transformations appliquées à l'image

Partie 1 — Préparation et Chargement

1.1 Image utilisée

La photographie retenue est un portrait de deux Cane Corsos (race de chien molossoïde). Elle offre une grande richesse de détails : pelage noir avec micro-textures, yeux ambrés très nets, rides du museau, arrière-plan flou (bokeh) et contours contrastés — autant d'éléments qui rendent chaque transformation particulièrement lisible.



Figure 1 — Image originale : atelier_image.jpg (612 × 408 px, mode RGB)

1.2 Chargement de l'image en Python

Après avoir placé le fichier image dans le même répertoire que le script, on l'ouvre avec PIL/Pillow et on le convertit immédiatement en tableau NumPy :

```
from scipy import ndimage
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# Chargement : PIL ouvre le fichier JPEG, np.array() le convertit en tenseur 3-D
image = np.array(Image.open('atelier_image.jpg'))

# Affichage de l'image originale
plt.figure(figsize=(8, 5))
plt.imshow(image)
plt.title('Image Originale')
plt.axis('off')
plt.show()

# Dimensions du tableau
print('image.shape :', image.shape) # --> (408, 612, 3)
print('image.dtype :', image.dtype) # --> uint8
```

1.3 Explication des dimensions — image.shape

Résultat obtenu : `image.shape = (408, 612, 3)`

Axe 0 : 408 → Hauteur de l'image en pixels (lignes, de haut en bas)

Axe 1 : 612 → Largeur de l'image en pixels (colonnes, de gauche à droite)

Axe 2 : 3 → Nombre de canaux couleur (Rouge, Vert, Bleu)

NumPy représente une image couleur comme un tenseur de rang 3 (tableau à 3 dimensions) de type `uint8` — entier non signé sur 8 bits, donc des valeurs entières de 0 à 255. Chaque position (ligne, colonne) contient un triplet de valeurs [R, G, B].

Canal (axe 2)	Couleur	Val. min (0)	Val. max (255)
0	Rouge (R)	Absence de rouge → noir	Rouge pur
1	Vert (G)	Absence de vert → noir	Vert pur
2	Bleu (B)	Absence de bleu → noir	Bleu pur

Notre image possède 3 canaux : c'est une image en mode RGB (couleur). Une image en niveaux de gris aurait 1 canal ; une image avec transparence (RGBA) en aurait 4.

Partie 2 — Transformations scipy.ndimage

Cinq familles de transformations sont appliquées. Pour chacune, le code Python est présenté, suivi d'une comparaison visuelle avant/après, puis d'une explication mathématique rigoureuse.

Transformation 1 — Flou Gaussien `ndimage.gaussian_filter()`

Code Python

```
# sigma : écart-type du noyau gaussien
# Plus sigma est grand → plus la fenêtre de pondération est large → flou plus fort
gauss3 = ndimage.gaussian_filter(image, sigma=3) # flou modéré
gauss8 = ndimage.gaussian_filter(image, sigma=8) # flou fort
```

Comparaison visuelle

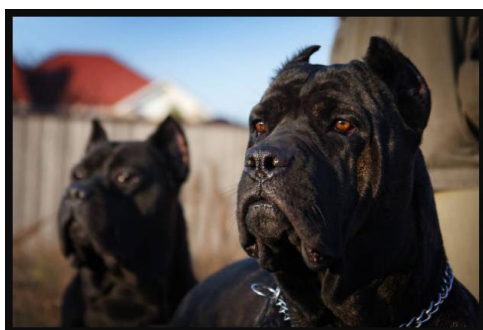
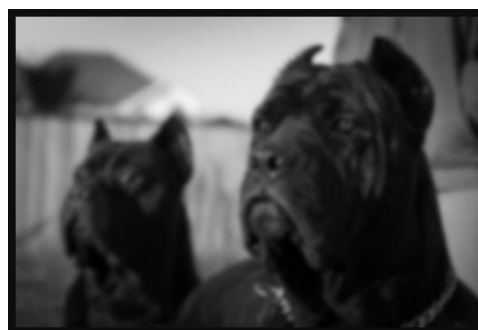


Image originale



Gaussien $\sigma = 3$ (modéré)

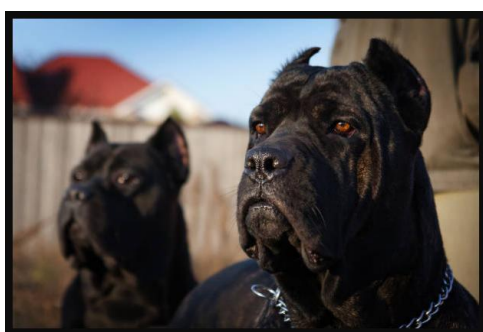
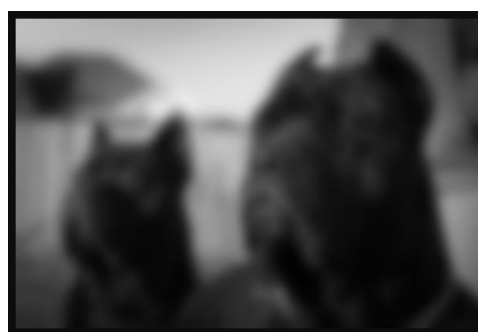


Image originale



Gaussien $\sigma = 8$ (fort)

Figure 2 — Flou gaussien : comparaison $\sigma = 3$ et $\sigma = 8$

Explication mathématique

Le filtre gaussien est une convolution 2-D entre l'image I et une fonction gaussienne bidimensionnelle G :

$$I'(x,y) = (I * G)(x,y)$$

Convolution : chaque pixel est la somme pondérée de ses voisins

$$G(x,y) = \frac{1}{2\pi\sigma^2} \cdot \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$$

Noyau gaussien 2-D centré sur (0,0), écart-type σ

Concrètement, chaque pixel de sortie reçoit une valeur calculée comme la moyenne pondérée de ses voisins, avec des poids qui décroissent selon une cloche gaussienne : les pixels proches ont un poids fort, les pixels éloignés un poids quasi nul. Cela produit un lissage progressif et naturel, sans artefact de bloc.

Observations sur la photo des Cane Corsos

$\sigma = 3$: les détails fins du pelage s'estompent légèrement, les yeux restent reconnaissables.

$\sigma = 8$: le pelage devient une masse noire uniforme, les rides du museau disparaissent presque.

Les hautes fréquences spatiales (textures fines) sont les premières atténuées.

Transformation 2 — Détection de contours (Sobel) `ndimage.sobel()`

Code Python

```
# Sobel requiert une image en niveaux de gris (tableau 2-D)
gray = image.mean(axis=2)          # moyenne des 3 canaux R,G,B

sob_x = ndimage.sobel(gray, axis=1) # gradient horizontal → bords verticaux
sob_y = ndimage.sobel(gray, axis=0) # gradient vertical   → bords
horizontal

# Magnitude totale du gradient
sob_mag = np.hypot(sob_x, sob_y)    # M = √(Gx² + Gy²)

# Normalisation 0-255 pour l'affichage
sob_norm = (sob_mag / sob_mag.max() * 255).astype(np.uint8)
```

Comparaison visuelle



Image originale (couleur)

Colormap gris — blanc = contour fort



Contours Sobel

Colormap gris — blanc = contour fort

Figure 3 — Détection de contours par le filtre de Sobel

Explication mathématique

Sobel approxime les dérivées partielles de l'intensité lumineuse par convolution avec deux noyaux discrets 3×3 :

$$K_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

Détection des variations horizontales (bords verticaux)

$$K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Détection des variations verticales (bords horizontaux)

$$G_x = I * K_x \quad | \quad G_y = I * K_y$$

Gradients obtenus par convolution

$$M(x,y) = \sqrt{G_x^2 + G_y^2}$$

Magnitude totale = force du contour au point (x,y)

La magnitude M est maximale là où l'intensité change brusquement (transition entre deux zones de luminosité différente) et quasi nulle dans les zones uniformes. C'est pourquoi les contours apparaissent en blanc sur fond noir.

Observations sur la photo des Cane Corsos

Les contours du museau, des oreilles et des yeux sont clairement détectés.

Le fond flou (bokeh) produit peu de gradient → zones sombres en arrière-plan.

Application directe : segmentation d'objets, détection de visages/corps en vision par ordinateur.

Transformation 3 — Rotation `ndimage.rotate()`

Code Python

```
# reshape=False → grille de sortie identique (peut rogner si l'image sort du cadre)
rot30 = ndimage.rotate(image, angle=30, reshape=False, mode='constant', cval=20)

# reshape=True → grille élargie pour contenir l'intégralité de l'image pivotée
rot45 = ndimage.rotate(image, angle=45, reshape=True, mode='constant', cval=20)

print(rot30.shape)  # --> (408, 612, 3)  ← grille inchangée
print(rot45.shape)  # --> (721, 721, 3)  ← grille élargie automatiquement
```

Comparaison visuelle

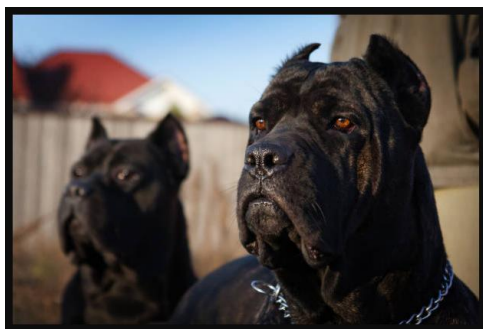
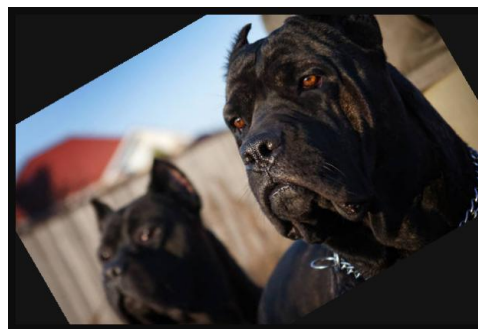


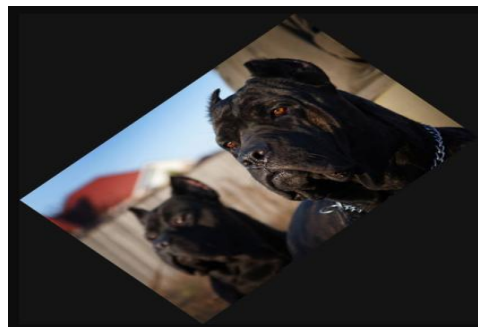
Image originale (408 × 612)



Rotation 30° — reshape=False



Image originale (408 × 612)



Rotation 45° — reshape=True (721 × 721)

Figure 4 — Rotation 30° (reshape=False) et 45° (reshape=True)

Explication mathématique

La rotation d'angle θ est une transformation linéaire orthogonale : elle préserve les distances et les angles. Pour chaque pixel source (x, y) , les coordonnées destination (x', y') sont :

$$|x'| = |\cos \theta \quad -\sin \theta| |x|$$

$$x' = x \cdot \cos \theta - y \cdot \sin \theta$$

$$|y'| = |\sin \theta \quad \cos \theta| |y|$$

$$y' = x \cdot \sin \theta + y \cdot \cos \theta$$

Les coordonnées destination étant des réels non entiers, SciPy applique une interpolation spline d'ordre 3 (bicubique) pour estimer la valeur du pixel à chaque position intermédiaire. `cval=20` remplit les zones hors-image d'un gris très sombre.

Observations sur la photo des Cane Corsos

reshape=False (30°) : les coins de la photo sont rognés, la grille reste (408×612).

reshape=True (45°) : la grille s'agrandit à (721×721) ≈ diagonale de l'originale.

La diagonale théorique : $\sqrt{(408^2+612^2)} \approx 734$ px — cohérent avec 721 px obtenu.

Transformation 4 — Zoom `ndimage.zoom()`

Code Python


```
# zoom = (facteur_hauteur, facteur_largeur, facteur_canaux)
# Le 3e facteur DOIT toujours valoir 1 pour ne pas toucher aux canaux couleur

zoom_in = ndimage.zoom(image, (2.0, 2.0, 1)) # agrandissement ×2
zoom_out = ndimage.zoom(image, (0.5, 0.5, 1)) # réduction ×0.5

print(zoom_in.shape) # --> (816, 1224, 3) : 408×2=816, 612×2=1224
print(zoom_out.shape) # --> (204, 306, 3) : 408×0.5=204, 612×0.5=306
```

Comparaison visuelle

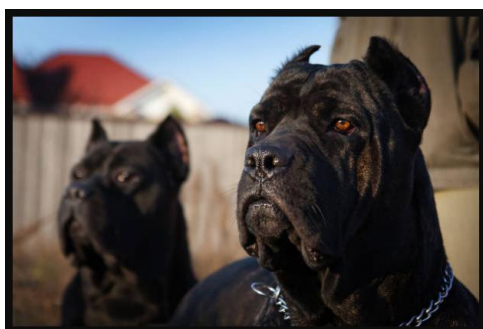
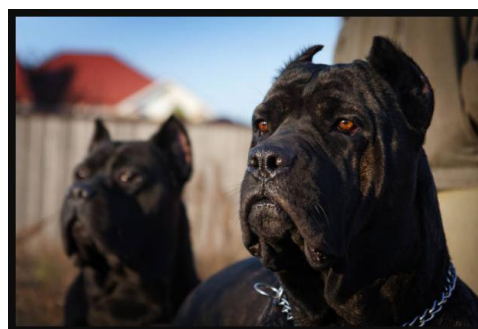


Image originale (408 × 612)



Zoom ×2 → (816 × 1224 px)

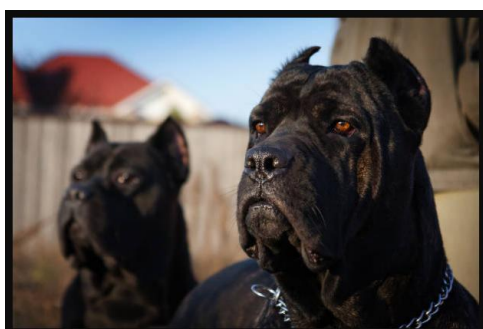
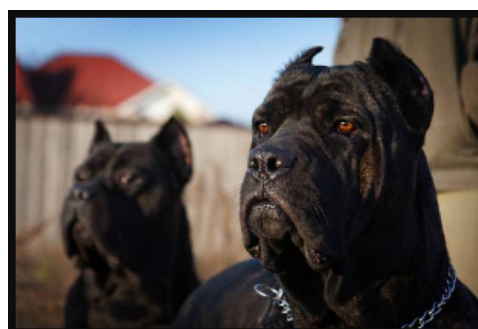


Image originale (408 × 612)



Zoom ×0.5 → (204 × 306 px)

Figure 5 — Zoom ×2 (agrandissement) et ×0.5 (réduction)

Explication mathématique

Le zoom est un rééchantillonnage spatial. Pour chaque coordonnée destination (xd, yd), on remonte à la coordonnée source correspondante, puis on interpole la valeur :

```
xs = xd / zoom_x | ys = yd / zoom_y
```

Coordonnées sources (nombres réels en général)

```
I'(xd, yd) = spline_interp(I, xs, ys)
```

Valeur estimée par interpolation spline (ordre 3)

```
dims_sortie = round(dims_entrée × zoom)
```

Nouvelles dimensions du tableau NumPy

L'interpolation spline cubique (ordre 3, par défaut) minimise les artefacts visuels. Pour un zoom très fort ($\times 4$ ou plus), un léger flou peut apparaître car l'algorithme ne peut pas inventer de détails absents de l'image source.

Observations sur la photo des Cane Corsos

Zoom $\times 2$: les chiens sont deux fois plus grands, les textures du pelage restent lisses.

Zoom $\times 0.5$: l'image est réduite de moitié ; les micro-détails du museau disparaissent.

Différence clé : `zoom()` modifie physiquement les dimensions du tableau NumPy.

Transformation 5 — Filtre de Moyenne `ndimage.uniform_filter()`

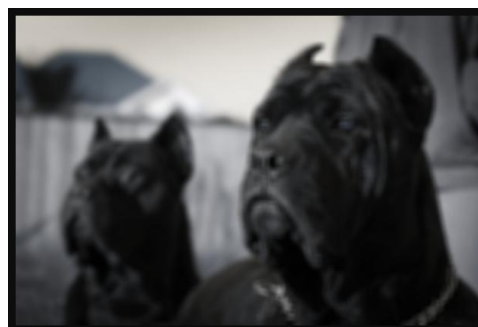
Code Python

```
# size = côté de la fenêtre carrée de moyennage (en pixels)
uni10 = ndimage.uniform_filter(image, size=10) # fenêtre 10×10
uni25 = ndimage.uniform_filter(image, size=25) # fenêtre 25×25
```

Comparaison visuelle



Image originale



Filtre Uniforme size=10

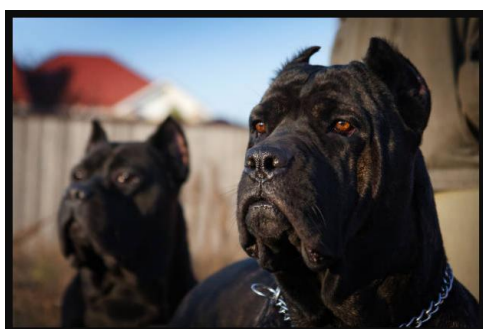
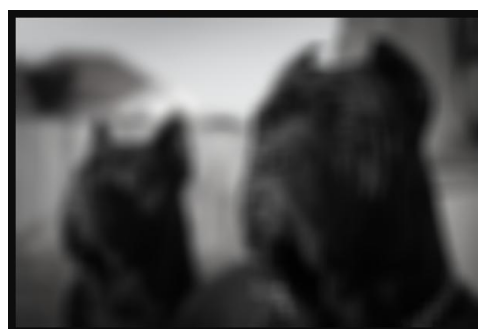


Image originale



Filtre Uniforme size=25 (fort)

Figure 6 — Filtre de Moyenne size=10 et size=25

Explication mathématique

Le filtre de moyenne (filtre uniforme ou box filter) remplace chaque pixel par la moyenne arithmétique de tous ses voisins dans une fenêtre carrée de côté n :

$$I'(x,y) = (1/n^2) \cdot \sum_i \sum_j I(x+i, y+j) \quad i, j \in [-\lfloor n/2 \rfloor ; \lfloor n/2 \rfloor]$$

Équivalent à : $I' = I * U$

$U = (1/n^2) \cdot \text{ones}(n,n)$ — noyau uniforme

Tous les voisins ont exactement le même poids $1/n^2$. C'est la distinction fondamentale avec le filtre gaussien où le poids décroît avec la distance au centre. Le filtre uniforme est donc plus « brutal » visuellement (léger artefact de halo ou de bloc aux bordures nettes), mais aussi plus rapide à calculer (pas d'exponentielle).

Comparaison Gaussien vs Filtre Uniforme

Gaussien : poids décroissant (naturel, préserve mieux les contours importants).

Uniforme : poids identique pour tous les voisins (plus brutal, léger effet de halo).

Sur la photo : à size=25, les yeux ambrés perdent leur netteté mais restent visibles.

Avantage pratique : calcul $\approx 3\times$ plus rapide que le gaussien → utile en temps réel.

Synthèse Comparative

Tableau récapitulatif des 5 transformations

Transformation	Fonction SciPy	Principe mathématique	Effet visuel	Paramètre clé
Flou Gaussien	gaussian_filter()	Convolution avec $G(x,y)=e^{-(x^2+y^2)/2\sigma^2}$	Lissage naturel	sigma
Contours Sobel	sobel()	Gradient: $M=\sqrt{G_x^2+G_y^2}$	Détecte les bords	axis
Rotation	rotate()	Transformation affine + interp. spline	Pivote l'image	angle
Zoom	zoom()	Rééchantillonnage: $x_s = x_d / \text{zoom}$	Agrandit / réduit	zoom
Filtre Uniforme	uniform_filter()	Moyenne $(1/n^2) \cdot \sum I(x+i,y+j)$	Lissage uniforme	size

Figure récapitulative — Toutes les transformations

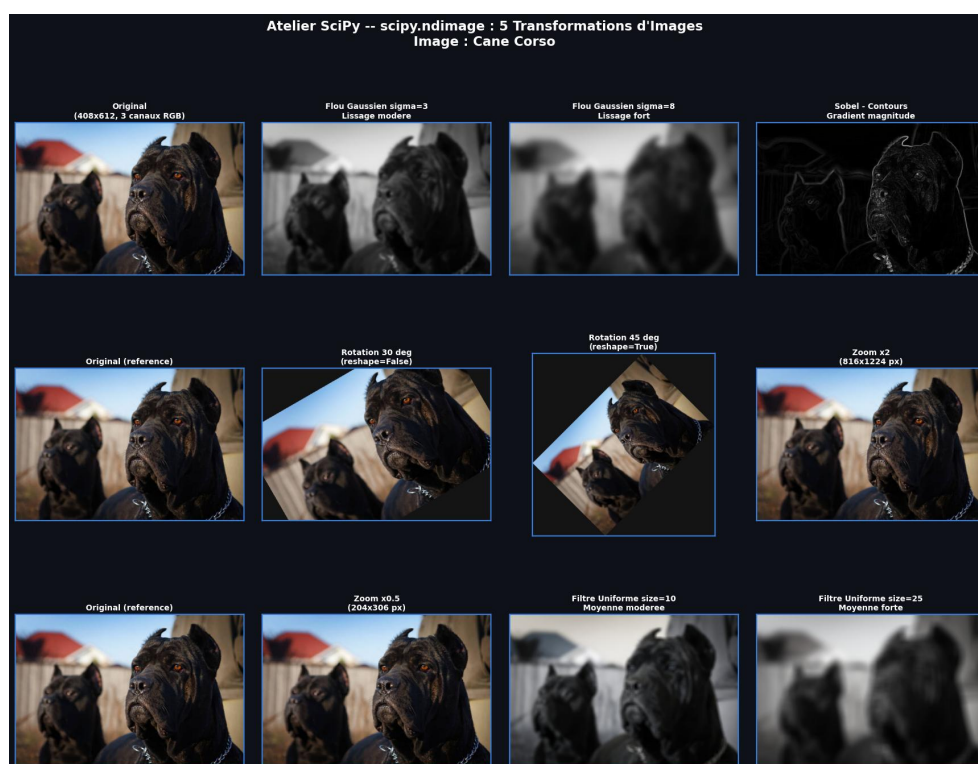


Figure 7 — Synthèse : image originale + 5 transformations (2 intensités chacune)

Conclusion

Cet atelier a permis d'explorer cinq transformations fondamentales du traitement numérique d'images grâce au module `scipy.ndimage`. Ces opérations se regroupent en deux grandes familles :

- **Opérations spectrales / de filtrage — modifient les valeurs des pixels**
 - Flou gaussien : convolution pondérée, atténue les hautes fréquences spatiales.
 - Filtre de moyenne : convolution uniforme, lissage brut par moyenne locale.
 - Sobel : dérivée partielle, met en évidence les zones de fort gradient (contours).
- **Opérations géométriques — modifient les positions des pixels**
 - Rotation : transformation affine linéaire + interpolation spline bicubique.
 - Zoom : rééchantillonnage spatial avec interpolation spline, modifie les dimensions du tableau.

La photo de Cane Corsos s'est révélée particulièrement pertinente pour illustrer chaque transformation : le pelage noir très texturé met en valeur les filtres de lissage, les contours nets du museau et des oreilles permettent une détection Sobel efficace, et la profondeur de champ

naturelle de la photographie offre des zones de référence à la fois très détaillées (premier plan) et uniformes (arrière-plan flou).

Ces opérations constituent les briques de base de la vision par ordinateur et du deep learning. Dans les réseaux de neurones convolutifs (CNN), les couches de convolution apprennent automatiquement des filtres analogues au Sobel ou au Gaussien. Le zoom et la rotation sont massivement utilisés en data augmentation pour enrichir les jeux d'entraînement.

Livrables rendus

- `atelier_scipy_ndimage.py` — Script Python complet et commenté
- `compte_rendu_scipy.docx` — Ce compte-rendu (explications + visuels)
- `atelier_image.jpg` — Image source : les deux Cane Corsos